Chapter 2 – Functional Dependencies (The Heart of Normalization)

Chapter Objectives

After completing this chapter, you will understand:

- What is a Functional Dependency (FD)?
- Why Functional Dependencies exist
- Determinants
- Candidate Keys
- Super Keys
- Primary Keys
- Alternate Keys
- Composite Keys
- Foreign Keys
- Prime & Non-prime Attributes
- Full Functional Dependency
- Partial Dependency
- Transitive Dependency
- Armstrong's Axioms (Introduction)
- Closure of Attributes
- How to identify dependencies
- Real-world examples
- Interview questions
- Common mistakes

---

1. Why Do We Need Functional Dependencies?

Suppose we have this table.

Student Table

StudentID	Name	Age
101	Akshat	22

StudentID	Name	Age
102	Rahul	21

Ask yourself...

If I know

```
StudentID = 101
```

Can I find the student's name?

Yes.

Can I find the student's age?

Yes.

So

StudentID determines

- Name
- Age

This relationship is called a **Functional Dependency**.

Definition

A **Functional Dependency (FD)** is a relationship between two attributes where one attribute uniquely determines another.

Mathematically,

```
A → B
```

Read it as

> **A determines B**

or

> **B is functionally dependent on A**

Real-Life Example

Think about your Aadhaar Number.

If I know your Aadhaar Number, I can determine:

- Your Name
- Date of Birth
- Address

So,

```
AadhaarNo → Name
AadhaarNo → DOB
AadhaarNo → Address
```

But...

If I know your Name,

Can I determine your Aadhaar Number?

No.

Because many people can have the same name.

So

```
Name → AadhaarNo
```

✗ Not true.

Important Rule

A Functional Dependency is **not about what happens in one sample of data**.

It is about a **business rule**.

For example:

StudentID	Name
101	Akshat
102	Rahul

Here:

```
StudentID → Name
```

is true because **every StudentID belongs to exactly one student**.

---

2. Determinant

In

```
A → B
```

The left side (A) is called the **Determinant**.

Example

```
StudentID → Name
```

Determinant = StudentID

Dependent Attribute = Name

Easy Memory Trick

Think of the determinant as a **question**.

`StudentID = 101`

Question asked.

Answer received.

`Name = Akshat`

The question determines the answer.

---

3. Candidate Key

A **Candidate Key** is the smallest set of attributes that can uniquely identify every row.

Example

StudentID	Email	Name
101	a@gmail.com	Akshat
102	b@gmail.com	Rahul

Can StudentID identify a row?

Yes.

Can Email identify a row?

Yes.

So

Both are Candidate Keys.

Why "Candidate"?

Because the database designer can choose any one of them as the Primary Key.

Example

Candidate Keys

`StudentID`

`Email`

Choose StudentID

↓

Primary Key

Email becomes

↓

Alternate Key

---

4. Primary Key

A **Primary Key** is the Candidate Key selected to uniquely identify each row.

Rules

- Cannot contain NULL.
- Must be unique.
- Only one Primary Key per table.
- Can contain one or more columns.

Example

StudentID (PK)	Name
101	Akshat
102	Rahul

---

5. Alternate Key

A Candidate Key that is **not chosen** as the Primary Key.

Example

Candidate Keys

`StudentID`

`Email`

Primary Key = StudentID

Alternate Key = Email

---

6. Super Key

A **Super Key** is any set of attributes that uniquely identifies a row.

It may contain **extra attributes**.

Example

StudentID	Name	Email

Super Keys

`StudentID`

`StudentID, Name`

`StudentID, Email`

`StudentID, Name, Email`

All identify the row.

But only the **smallest** one is the Candidate Key.

Difference Between Super Key and Candidate Key

Suppose

`StudentID`

already identifies every row.

Then

`StudentID + Name`

also identifies every row.

But Name is unnecessary.

So

`StudentID + Name`

is a Super Key

not

Candidate Key.

Memory Trick

Super Key

May contain extra attributes.

Candidate Key

No unnecessary attributes.

---

7. Composite Key

A Primary Key made from **more than one column**.

Example

Enrollment Table

StudentID	CourseID
101	C1
101	C2

Can StudentID identify rows?

No.

Can CourseID identify rows?

No.

Together?

Yes.

Primary Key

`(StudentID, CourseID)`

This is called a Composite Key.

---

8. Foreign Key

A Foreign Key creates a relationship between two tables.

Student Table

StudentID	Name
101	Akshat

Enrollment Table

StudentID	Course
101	SQL

StudentID inside Enrollment refers to Student table.

Therefore

StudentID is a Foreign Key.

Why Foreign Keys?

To maintain **Referential Integrity**.

Example

Student Table

StudentID
101
102

Enrollment

StudentID
999

Student 999 doesn't exist.

This should not happen.

Foreign Keys prevent such invalid references.

---

9. Prime Attribute

An attribute that is part of **any Candidate Key**.

Example

Candidate Key

`(StudentID, CourseID)`

Prime Attributes

- StudentID
- CourseID

---

10. Non-prime Attribute

Attributes that are **not part of any Candidate Key**.

Example

StudentID CourseID StudentName CourseName

Candidate Key

`(StudentID, CourseID)`

Prime

- StudentID
- CourseID

Non-prime

- StudentName
- CourseName

---

11. Full Functional Dependency

Suppose

Primary Key

`(StudentID, CourseID)`

Now

`(StudentID, CourseID) → Marks`

Can StudentID alone determine Marks?

No.

Can CourseID alone determine Marks?

No.

Need both.

This is

Full Functional Dependency.

---

12. Partial Dependency

Suppose

`(StudentID, CourseID)`

Now

`StudentID → StudentName`

Only StudentID is enough.

CourseID isn't needed.

This is

Partial Dependency.

This is exactly what **2NF removes**.

---

13. Transitive Dependency

Suppose

`StudentID → DepartmentID`

and

`DepartmentID → DepartmentName`

Therefore

`StudentID → DepartmentName`

through DepartmentID.

This is

Transitive Dependency.

This is what **3NF removes**.

Dependency Chain Visualization

```
StudentID
     ↓
DepartmentID
     ↓
DepartmentName
```

DepartmentName depends on another non-key attribute.

Not allowed in 3NF.

---

14. Real-Life Example

Employee Table

EmployeeID	DepartmentID	DepartmentName
1	D1	HR

Dependencies

```
EmployeeID → DepartmentID
```

```
DepartmentID → DepartmentName
```

Not 3NF.

---

15. Armstrong's Axioms (Introduction)

These are rules used to derive Functional Dependencies.

Reflexivity

If

```
A contains B
```

then

```
A → B
```

Example

```
(StudentID, Name)
```

```
↓
```

```
(StudentID, Name) → StudentID
```

Always true.

Augmentation

If

```
A → B
```

Then

```
AC → BC
```

Example

```
StudentID → Name
```

Then

```
(StudentID, CourseID)

↓

(Name, CourseID)
```

Also true.

Transitivity

If

```
A → B

B → C
```

Then

```
A → C
```

Example

```
StudentID → DepartmentID

DepartmentID → DepartmentName
```

Then

```
StudentID → DepartmentName
```

---

16. Attribute Closure (Very Important)

Suppose

```
A → B

B → C
```

Find

```
A+
```

Start

```
A+
=
{A}
```

Using

```
A → B
```

Now

`{A,B}`

Using

`B → C`

Now

`{A,B,C}`

Therefore

```
A+
=
{A,B,C}
```

Attribute closure helps determine whether an attribute (or set of attributes) is a candidate key.

---

17. How to Find Functional Dependencies

Ask this question:

> **If I know X, can I always determine Y?**

Examples:

- StudentID → Name ✅
- Email → StudentID (if emails are unique) ✅
- Name → StudentID ❌ (many students can have the same name)
- CourseID → CourseName ✅

---

18. Common Mistakes

❌ Mistake 1

Thinking every unique value is a Functional Dependency.

Wrong.

Functional Dependencies come from **business rules**, not just sample data.

❌ Mistake 2

Confusing Candidate Key with Primary Key.

Candidate Keys = all possible unique identifiers.

Primary Key = the one selected.

❌ Mistake 3

Thinking every Super Key is a Candidate Key.

Wrong.

Super Keys may contain unnecessary attributes.

---

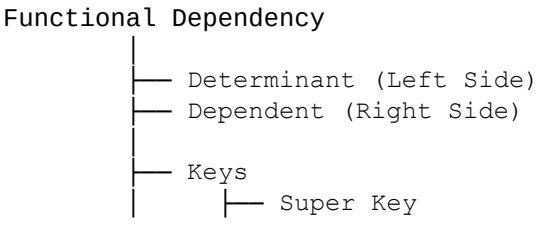
19. Summary Table

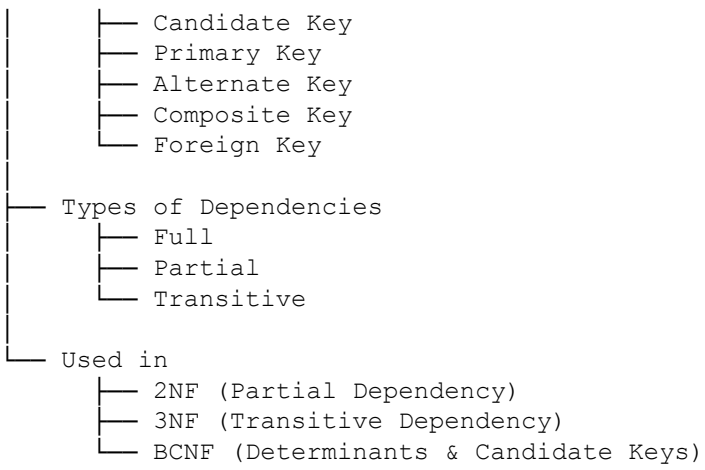
Concept	Meaning
Functional Dependency	One attribute determines another
Determinant	Left side of the dependency
Candidate Key	Smallest unique identifier
Primary Key	Chosen candidate key
Alternate Key	Candidate key not selected
Super Key	Unique identifier (may have extra attributes)
Composite Key	Key made from multiple columns
Foreign Key	References another table
Prime Attribute	Part of a candidate key
Non-prime Attribute	Not part of any candidate key
Full Dependency	Depends on the whole composite key
Partial Dependency	Depends on part of the composite key
Transitive Dependency	Depends on another non-key attribute

Interview Questions

1. What is a Functional Dependency?
2. What is a determinant?
3. Differentiate between Super Key and Candidate Key.
4. Can a table have multiple Candidate Keys?
5. Can a table have multiple Primary Keys?
6. What is a Composite Key?
7. What is a Foreign Key?
8. Explain Full, Partial, and Transitive Dependency with examples.
9. What is a Prime Attribute?
10. What is Attribute Closure, and why is it important?

Chapter Memory Map




Candidate Key
Primary Key
Alternate Key
Composite Key
Foreign Key
Types of Dependencies
Full
Partial
Transitive
Used in
2NF (Partial Dependency)
3NF (Transitive Dependency)
BCNF (Determinants & Candidate Keys)